

Лабораторная работа № 6

Тема работы: Передача информации между процессами.

Цель работы: Изучить основы взаимодействия локальных процессов на базе файлов, проецируемых в память.

Изучаемые вопросы

1. Функции для работы с файлами, отображаемыми в память.
2. Файлы данных, проецируемые в память.
3. Совместный доступ процессов к данным через механизм проецирования.
4. Обработка текстового и графического файлов двумя процессами.
5. Работа с файлами больших размеров.
6. Взаимодействие процессов через Page файл.

Постановка задачи

1. Осуществить обмен текстовыми и графическими данными между двумя приложениями P1 и P2 с выводом информации в заданную область окна программы.
2. Реализовать обработку процессом файлов больших размеров.
3. Осуществить обмен данными между двумя приложениями P1 и P2 через Page файл с выводом информации в заданную область окна программы.

Теоретические сведения по изучению темы

Функции для работы с файлами, отображаемыми в память

Функция **CreateFile** позволяет процессу открывать файл, проецируемый в память другим процессом:

```
hTxtFile = CreateFileW(L"file123.txt",GENERIC_READ,FILE_SHARE_READ, NULL,OPEN_EXISTING,  
FILE_ATTRIBUTE_NORMAL, NULL);
```

Функция **CreateFileMapping** создаёт уже другие, «проецирующие» внутренние структуры и связывает «сущность на диске» с «сущностью в адресном пространстве»:

```
//Создание объекта ядра «проекция файла»  
hTxtMappingFile=CreateFileMappingA(hTxtFile,NULL,PAGE_READONLY,0,0,"file123");
```

По своему месту в последовательности действий она создаёт что-то наподобие «адресного пространства из файла». Этот объект поддерживает

соответствие между содержимым файла и адресным пространством процесса, использующего этот файл (“file123” – имя объекта файлового отображения). Так как мы хотим отобразить весь файл, то четвертый и пятый параметры должны быть равны нулю.

Функция **MapViewOfFile** делает "проекцию в память":

```
//Проецирование данных файла на адресное пространство процесса  
hTxtMapFileStartAddr = MapViewOfFile(hTxtMappingFile,FILE_MAP_READ,0,0,0);
```

Здесь hTxtMappingFile – дескриптор созданного объекта файлового отображения. Второй параметр определяет режим доступа к файлу. Значение, возвращаемое функцией MapViewOfFile, имеет тип "указатель". Если функция отработала успешно, то она вернет начальный адрес данных объекта файлового отображения.

Функция **OpenFileMapping** используется для открытия объекта ядра «проекция файла»:

```
//Открытие объекта ядра «проекция файла»  
hTxtMappingFile = OpenFileMappingA(FILE_MAP_READ,FALSE,"file123");
```

Функция **UnmapViewOfFile** прекращает отображение в адресное пространство процесса того файла, который перед этим был отображен при помощи функции **MapViewOfFile**:

```
UnmapViewOfFile(hTxtMapFileStartAddr);
```

Файлы данных, проецируемые в память

Операционная система позволяет проецировать на адресное пространство процесса и файл данных. Это очень удобно при манипуляциях с большими потоками данных.

Для проецирования файла данных нужно выполнить три операции:

1. Создать или открыть объект ядра "файл", идентифицирующий дисковый файл, который Вы хотите использовать как проецируемый в память. Для создания объекта “файл” используется функция **CreateFile**.

2. С помощью функции **CreateFileMapping** создается объект ядра “проецируемый файл”, чтобы сообщить системе размер файла и способ

доступа к нему. При этом используется описатель файла (handle), возвращенный функцией **CreateFile**. Теперь файл готов к проецированию.

3. Производится отображение объекта “проецируемый файл” или его части на адресное пространство процесса. Для этого применяется функция **MapViewOfFile**.

Для открепления файла от адресного пространства процесса используется функция **UnmapViewOfFile**, а для уничтожения объектов “файл” и “проецируемый файл” – функция **CloseHandle**.

Общая схема работы с проецированными файлами такова:

```
//Открытие объекта ядра «файл»  
hTxtFile = CreateFileW(L"file123.txt",...);  
//Создание объекта ядра «проекция файла»  
hTxtMappingFile=CreateFileMappingA(hTxtFile,...,"file123");  
//Проецирование данных файла на адресное пространство процесса  
hTxtMapFileStartAddr = MapViewOfFile(hTxtMappingFile,...);
```

Совместный доступ процессов к данным через механизм проецирования

Совместное использование данных в этом случае происходит так: два или более процесса проецируют в память представления одного и того же объекта "проекция файла", т.е. одни и те же страницы физической памяти. В результате, когда один процесс записывает данные в представление общего объекта "проекция файла", изменения немедленно отражаются на представлениях в других процессах. Но при этом все процессы должны использовать одинаковое имя объекта "проекция файла".

1.exe

```
case WM_INITDIALOG:
    hFileMapping = CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE, 0,
    1000, TEXT("text123text"));

    if(hFileMapping == NULL)
    {
        MessageBox(hDlg,TEXT("Невозможно создать проекцию файла в Page-
        файле..."), TEXT("Error"), MB_ICONERROR);
    }
    else
    {
        sharedBuffer = (LPTSTR) MapViewOfFile(hFileMapping, FILE_MAP_ALL_ACCESS, 0,
        0, 0);
    }
}
```

2.exe

```
hFileMapping = OpenFileMapping(FILE_MAP_READ,FALSE,TEXT("text123text"));

if(hFileMapping == NULL)
{
    MessageBox(hDlg,TEXT("Невозможно создать проекцию файла..."), TEXT("Error"),
    MB_ICONERROR);
}
else
{
    sharedBuffer = (LPTSTR) MapViewOfFile(hFileMapping, FILE_MAP_READ, 0, 0, 0);
    DWORD dwError = GetLastError();
}
}
```

Обработка текстового и графического файлов двумя процессами

При обработке текстовых и графических файлов небольшого размера можно спроецировать весь файл в память. Затем произвести его обработку. По завершении обработки файла необходимо прекратить отображение на адресное пространство представления объекта «проекция файла» и закрыть описатель всех объектов ядра.

Текстовый файл:

```
//Открытие объекта ядра «проекция файла»
hTxtMappingFile = OpenFileMappingA(FILE_MAP_READ,FALSE,"text123");
if(hTxtMappingFile != NULL)
{
//Проецирование данных файла на адресное пространство процесса
hTxtMapFileStartAddr = MapViewOfFile(hTxtMappingFile, FILE_MAP_READ, 0, 0, 0);
//Вывод данных из текстового файла в edittext
SetDlgItemTextA(hDlg, IDC_EDIT_TEXT, (char*)hTxtMapFileStartAddr);
//Отключение файла данных от адресного пространства процесса
UnmapViewOfFile(hTxtMapFileStartAddr);
//Закрытие объекта «проекция файла»
CloseHandle(hTxtMappingFile);
```

Графический файл:

```
//Открытие объекта ядра «файл»
hBmpFile = CreateFileA("image123.bmp", GENERIC_READ, FILE_SHARE_READ, NULL,
OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
GetClientRect(GetDlgItem(hDlg, IDC_BMPFRAME), &rect);
//Создание объекта ядра «проекция файла»
hBmpMappingFile = CreateFileMappingA(
    hBmpFile,                //описатель файла
    NULL,                    //аттрибуты защиты объекта ядра
    PAGE_READONLY,           //атрибут защиты, присваиваемый
    // страницам физической памяти
    0,0,                     //максимальный размер файла в
    // байтах (0 -- размер меньше 4Гб)
    "BmpFile");              //имя объекта ядра
//Проецирование данных файла на адресное пространство процесса
hBmpMapFileAddr = MapViewOfFile(
    hBmpMappingFile,         // описатель объекта «проекция файла»
    FILE_MAP_READ,          // вид доступа к данным
```

```

0,0, // смещение в файле до байта файла данных,
//который нужно считать в представлении первым
0); // сколько байтов файла данных должно быть
//спроецировано на адресное пространство
//(0 -- от смещения и до конца файла)
BITMAPFILEHEADER *bFileHeader= (BITMAPFILEHEADER*)hBmpMapFileAddr;
BITMAPINFO *bInfo=(BITMAPINFO*)((char*)hBmpMapFileAddr+14);
hdc=GetDC(GetDlgItem(hDlg, IDC_BMPFRAME));
hBmpFile=CreateDIBitmap(hdc,&(bInfo->bmiHeader), CBM_INIT, (char*)hBmpMapFileAddr +
bFileHeader->bfOffBits, bInfo,DIB_PAL_COLORS);
hMemDC=CreateCompatibleDC(hdc);
SelectObject(hMemDC,hBmpFile);
StretchBlt(hdc, 0, 0, rect.right, rect.bottom, hMemDC, 0, 0, bInfo->bmiHeader.biWidth, bInfo->
bmiHeader.biHeight, SRCCOPY);
ReleaseDC(GetDlgItem(hDlg, IDC_BMPFRAME), hdc);
DeleteDC(hMemDC);
DeleteObject(hBmpFile);
return (INT_PTR)TRUE;

```

Работа с файлами больших размеров

```

//*****Работа с большим файлом*****
if (LOWORD(wParam) == IDC_HUGE)
{
    // начальные границы представлений всегда начинаются по адресам,
    // кратным гранулярности выделения памяти
    SYSTEM_INFO sinf;
    GetSystemInfo(&sinf);
    // открываем файл данных
    HANDLE hBigFile = CreateFileA("TEST.txt", GENERIC_READ, FILE_SHARE_READ, NULL,
    OPEN_EXISTING, FILE_FLAG_SEQUENTIAL_SCAN, NULL);
    // создаем объект проекции файла
    HANDLE hHugeFileMapping = CreateFileMapping(hBigFile, NULL,PAGE_READONLY, 0, 0,
    NULL);
    DWORD dwFileSizeHigh;
    __int64 dwFileSize = GetFileSize(hBigFile, &dwFileSizeHigh);
    dwFileSize += (((__int64) dwFileSizeHigh) << 32);
    // доступ к описателю объекта файл нам больше не нужен
    CloseHandle(hBigFile);
    __int64 dwFileOffset = 0;
    DWORD dwNumOfOs = 0;
    char symbol[100] = "";
}

```

```

DWORD dwPrBarRange = dwFileSize / sinf.dwAllocationGranularity;

while (dwFileSize > 0)
{
    // определяем, сколько байтов надо спроецировать
    DWORD dwBytesInBlock = sinf.dwAllocationGranularity;
    if (dwFileSize < sinf.dwAllocationGranularity)
        dwBytesInBlock = (DWORD)dwFileSize;
    //Проецирование данных файла на адресное пространство процесса
    PBYTE hHugeMapFileStartAddr = (PBYTE)
    MapViewOfFile(hHugeFileMapping, FILE_MAP_READ,
        (DWORD) (dwFileOffset >> 32), // начальный байт
        (DWORD) (dwFileOffset & 0xFFFFFFFF), // в файле
        dwBytesInBlock); // число проецируемых байтов
    SetDlgItemTextA(hDlg, IDC_EDIT_HUGEFILE, (char*) hHugeMapFileStartAddr);
    SendMessage(GetDlgItem(hDlg, IDC_PROGRESS),
        (UINT)PBM_STEPIT, 0, 0);
    // прекращаем проецирование представления,
    // чтобы в адресном пространстве
    // не образовалось несколько представлений одного файла

    UnmapViewOfFile(hHugeMapFileStartAddr);

    // переходим к следующей группе байтов в файле
    dwFileOffset += dwBytesInBlock;
    dwFileSize -= dwBytesInBlock;
}
CloseHandle(hHugeFileMapping);
return (INT_PTR)TRUE;
}

```

Пример взаимодействия процессов через Page файл.

1.exe

case WM_INITDIALOG:

```
hFileMapping = CreateFileMapping(INVALID_HANDLE_VALUE, NULL, PAGE_READWRITE,
0, 1000, TEXT("text123text"));
if(hFileMapping == NULL)
{
    MessageBox(hDlg,TEXT("Невозможно создать проекцию файла в Page-
файле..."), TEXT("Error"), MB_ICONERROR);
}
else
{
    sharedBuffer = (LPTSTR) MapViewOfFile(hFileMapping, FILE_MAP_ALL_ACCESS, 0, 0, 0);
}
```

2.exe

```
hFileMapping = OpenFileMapping(FILE_MAP_READ, FALSE, TEXT("text123text"));
if(hFileMapping == NULL)
{
    MessageBox(hDlg,TEXT("Невозможно создать проекцию файла..."),
    TEXT("Error"), MB_ICONERROR);
}
else
{
    sharedBuffer = (LPTSTR) MapViewOfFile(hFileMapping,
    FILE_MAP_READ,0,0,0);
    DWORD dwError = GetLastError();
}
```